

Classes and Objects

Programming and Data Structures — Week 1, Lecture 1

Dr. Innocent Nyalala

Why we start here, not with data structures

Every structure in this course — stack, queue, linked list, tree, graph — is built as a **Python class**. If the class mechanism is shaky, every week after this one is shaky too.

This lecture has one goal: by the end, you can look at any `class` block in this course's textbook and explain what happens when it runs.

The problem, before the solution

Suppose you are tracking points on a 2D plane and need the distance between two of them. Without a class, you might reach for a tuple:

```
p1 = (0, 0)
p2 = (3, 4)
distance = ((p1[0] - p2[0]) ** 2 + (p1[1] - p2[1]) ** 2) ** 0.5
```

This works, but `p1[0]` and `p1[1]` mean nothing without a comment, and nothing stops you from writing `p1 = (0, 0, 0)` by mistake — a 3-tuple that silently breaks this formula three lines later, far from where the bug actually is.

A class is a blueprint

```
1 class Point:
2     """A point in the 2D plane."""
3
4     def __init__(self, x, y):          # runs once, when a Point is created
5         self.x = x                    # self.x is THIS point's x, not some
```

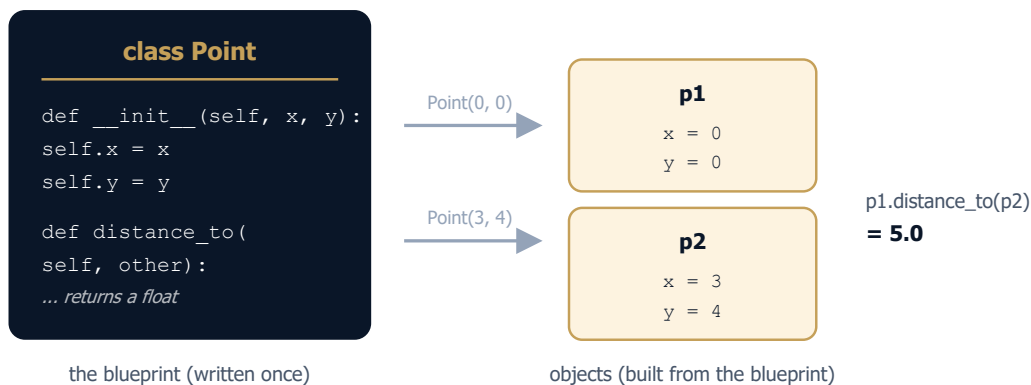
```

6         self.y = y                                # global variable named x
7
8     def distance_to(self, other):
9         dx = self.x - other.x
10        dy = self.y - other.y
11        return (dx ** 2 + dy ** 2) ** 0.5

```

class Point: declares the blueprint. `__init__` is the constructor — Python calls it automatically the moment you write `Point(0, 0)`. `self` is how the method reaches back into the specific object it was called on.

Class vs. object, drawn out



The class is written once. `Point(0, 0)` and `Point(3, 4)` are two different **objects**, each with its own `x` and `y`, both built from the same blueprint.

Running it

```

p1 = Point(0, 0)
p2 = Point(3, 4)
print(p1.distance_to(p2))

```

5.0

Read `p1.distance_to(p2)` as “ask `p1` to compute its distance to `p2`.” `self` inside the method is `p1`; `other` is `p2`. Nothing here is special syntax — it is the same function-call mechanism as always, with the object silently passed as the first argument.

Attributes vs. methods

	Holds data	Holds behavior
Attribute	<code>self.x</code> , <code>self.y</code>	—
Method	—	<code>distance_to(self, other)</code>

An attribute is a value stored on the object. A method is a function stored on the class, called through an object. Every method’s first parameter is `self` by convention — Python passes the object there automatically; you never write it yourself when calling.

Common first mistake

```
class Point:
    def __init__(self, x, y):
        x = x          # BUG: assigns to the local parameter, not self.x
        y = y

p = Point(3, 4)
print(p.x)           # AttributeError: 'Point' object has no attribute 'x'
```

Forgetting `self.` on the left-hand side is the single most common bug in a first `__init__`. `x = x` does nothing useful — it reassigns the local parameter `x` to itself and throws the value away. The object never gets an `x` attribute at all.

Live coding exercise (5 minutes)

Extend `Point` with a `midpoint(self, other)` method that returns a new `Point` sitting exactly between `self` and `other`.

```
def midpoint(self, other):
    return Point((self.x + other.x) / 2,
                 (self.y + other.y) / 2)
```

Notice the method **returns a new** `Point`, built with the same `Point(...)` constructor you already have. Methods can and often do construct more objects of their own class.

Before next lecture

You now have: a class with a constructor, attributes, and a method that takes another object as an argument. Next lecture: what happens when we let data be changed carelessly, and how a class can refuse to allow it — encapsulation.